

香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Operating Systems
Lecture 8

Synchronization:
Lock implementation and semaphore

Prof. Yunming XIAO

School of Data Science

Acknowledgments: Ion Stoica and Matei Zaharia, Berkeley CS 162

Recall: correctness with concurrency is hard

- Threaded programs must work for all interleavings of thread instruction sequences
- Cooperating threads are inherently non-deterministic and non-reproducible
- Really hard to debug unless carefully designed!

The milk purchase problem



Motivating example: “too much milk”

Time	Person A	Person B
3:00	Look in Fridge. Out of milk	
3:05	Leave for store	
3:10	Arrive at store	Look in Fridge. Out of milk
3:15	Buy milk	Leave for store
3:20	Arrive home, put milk away	Arrive at store
3:25		Buy milk
3:30		Arrive home, put milk away

Too much milk: correctness properties

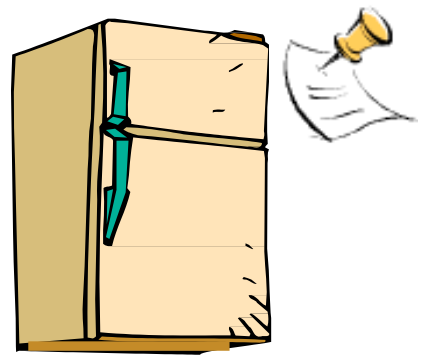
- What are the correctness properties for the “Too much milk” problem???
  - Never more than one person buys
  - Someone buys if needed

- First attempt: restrict ourselves to use only atomic load and store operations as building blocks

Too much milk: solution #1

- Use a note to avoid buying too much milk:
  - Leave a note before buying (kind of “lock”)
  - Remove note after buying (kind of “unlock”)
  - Don’t buy if note (wait)
- Suppose a computer tries this (remember, only memory read/write are atomic)

```
if (no_milk) {
  if (no_note) {
    leave Note;
    buy milk;
    remove note;
  }
}
```



Too much milk: solution #1

Thread 1

```
if (no_milk) {


  if (no_note) {
    leave note;
    buy milk;
    remove note;
  }
}
```

Thread 2

```
if (no_milk) {
  if (no_note) {

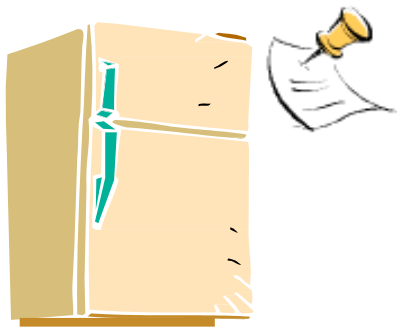

    leave note;
    buy milk;
    remove note;
  }
}
```

Too much milk: solution #1

- Still too much milk but only occasionally!

- Thread can get context switched after checking milk and note but before buying milk!

- Solution makes problem worse since fails intermittently
  - Makes it really hard to debug…
  - Must work despite what the scheduler does!



Too much milk: solution #1½

- Let's try to fix this by placing note first

```
leave note;
if (no_milk) {
  if (no_note) {
    buy milk;
  }
}
remove note;
```

- What happens here?
  - Well, with human, probably nothing bad
  - With computer: no one ever buys milk

Too much milk: solution #2

- How about labeled notes?
  - Now we can leave note before checking

- Algorithm looks like this:

Thread 1

```
leave note_A;
if (no_note_B) {
  if (no_milk) {
    buy milk;
  }
}
remove note_A;
```

Thread 2

```
leave note_B;
if (no_note_A) {
  if (no_milk) {
    buy milk;
  }
}
remote note_B;
```

Too much milk: solution #2

- Possible for neither thread to buy milk
  - Context switches at exactly the wrong times can lead each to think that the other is going to buy

- Really insidious:
  - Extremely unlikely this would happen, but will at worst possible time
  - Probably something like this in UNIX

Too much milk: solution #3 (Peterson's algorithm)

Thread 1

```
leave note_A;
while (note_B) { //X
  do nothing;
}
if (no_milk) {
  buy milk;
}
remove note_A;
```

Thread 2

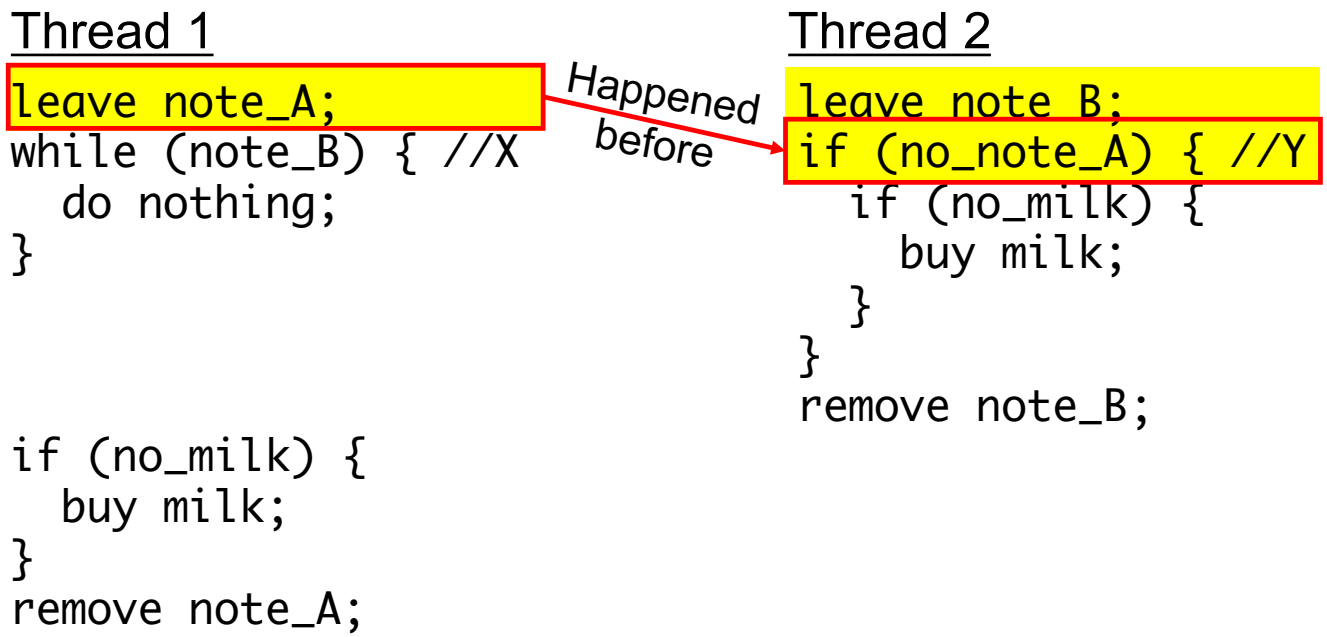
```
leave note_B;
if (no_note_A) { //Y
  if (no_milk) {
    buy milk;
  }
}
remove note_B;
```

To reason about whether this is correct

- Need some details about how our computer's memory works (a "memory model")
- We already assumed that individual loads and stores are atomic
- What can we assume about a series of loads and stores by our threads? In this example, we'll assume **sequential consistency**, which means:
  - There is a single, global order of loads and stores, and the operations issued by each thread are processed in program order
- For example, when Thread A does "buy milk" followed by "remove note A", Thread B will be able to "see" the milk before it "sees" that note A is gone

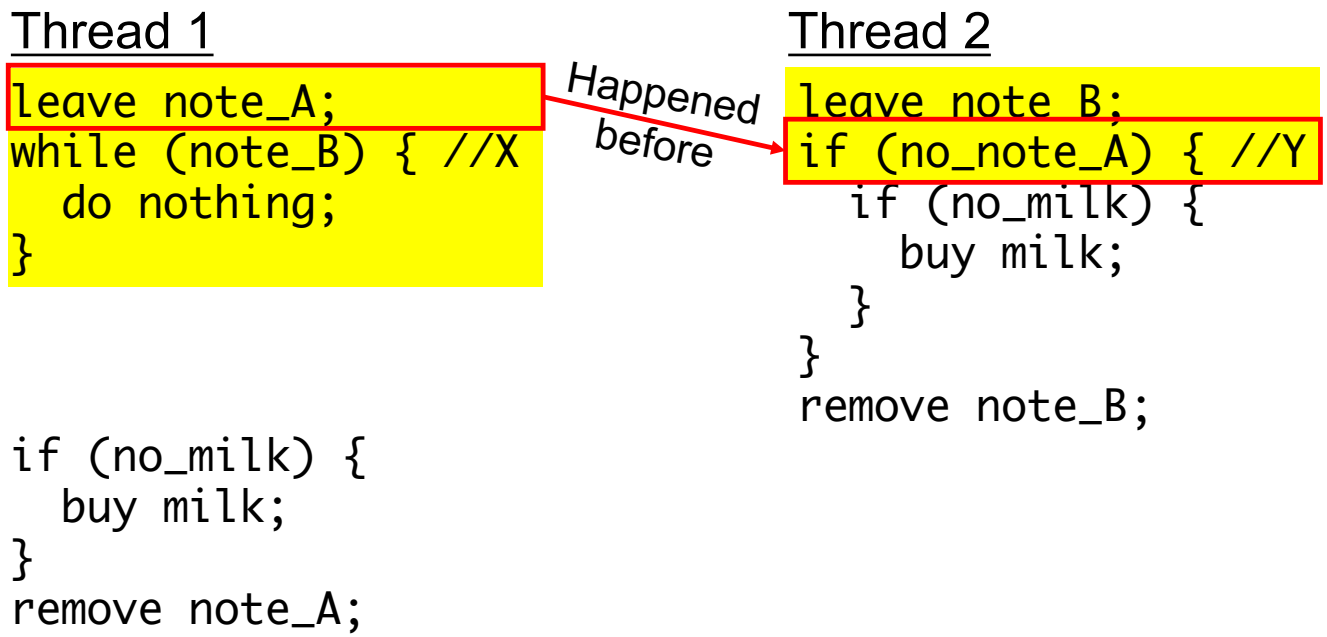
Case 1

- "`leave note_A`" happens before "`if (no_note_A)`"



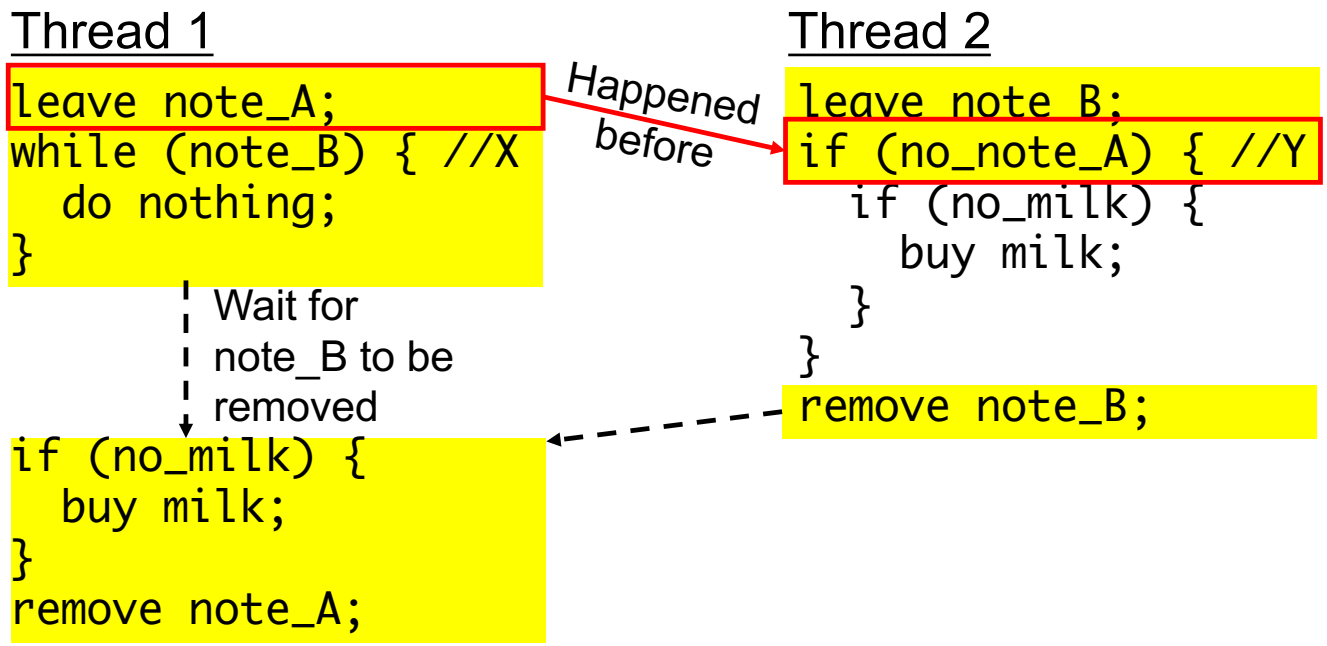
Case 1

- "`leave note_A`" happens before "`if (no_note_A)`"



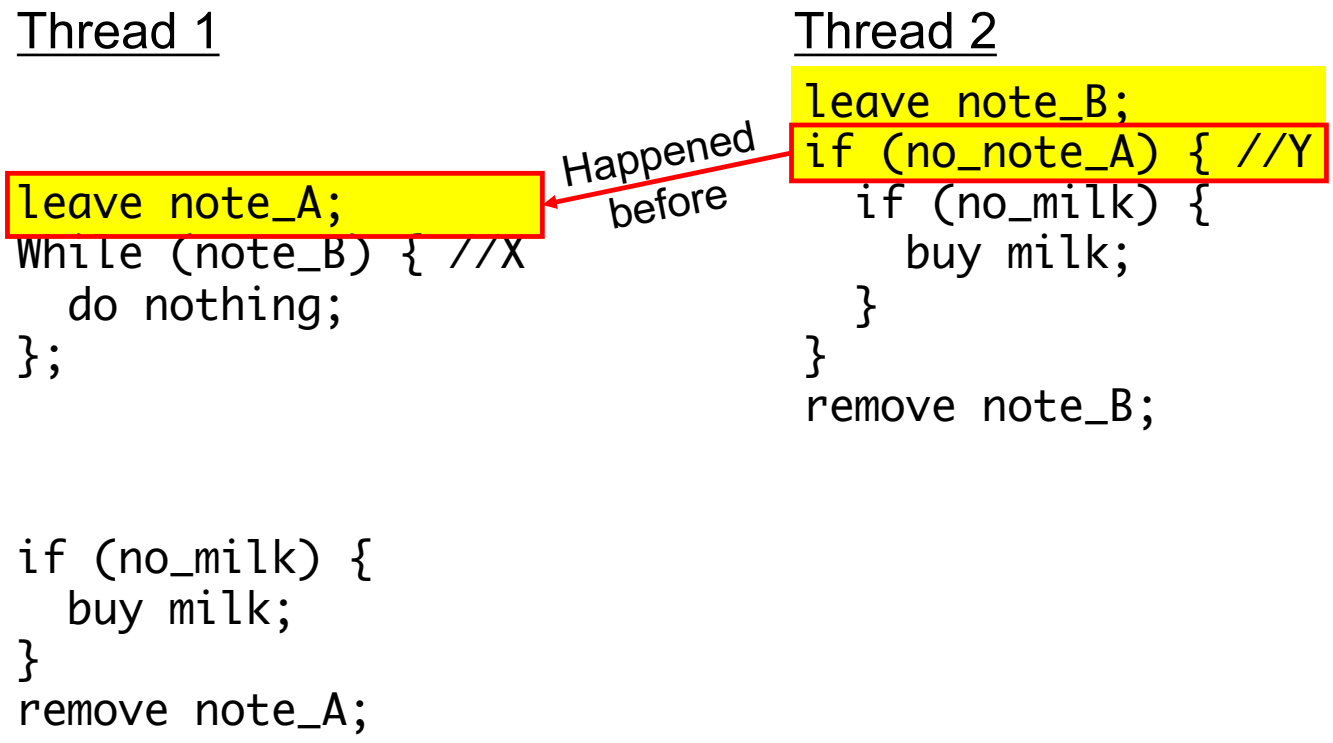
Case 1

- "`leave note_A`" happens before "`if (no_note_A)`"



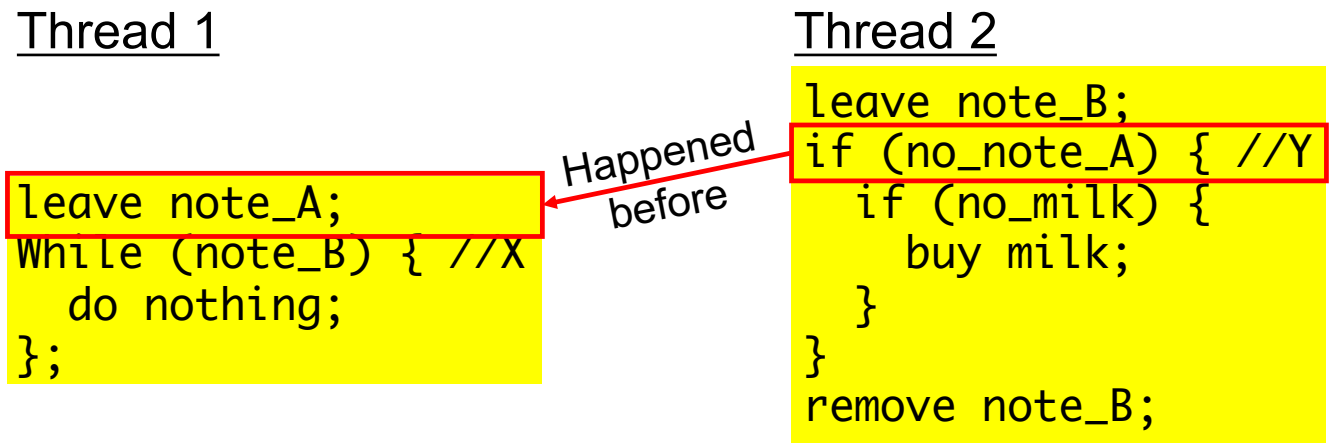
Case 2

- "`if (no_note_A)`" happens before "`leave note_A`"



Case 2

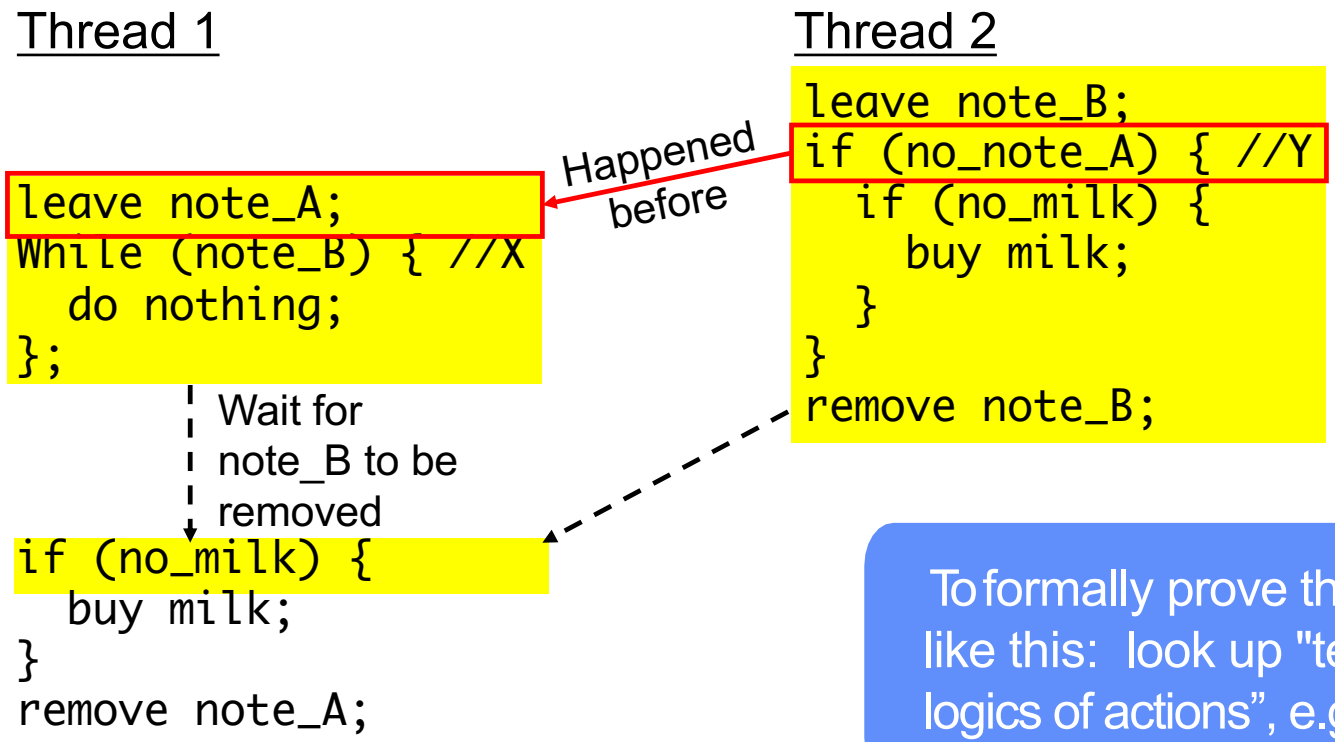
- "`if (no_note_A)`" happens before "`leave note_A`"




```
if (no_milk) {
  buy milk;
}
remove note_A;
```

Case 2

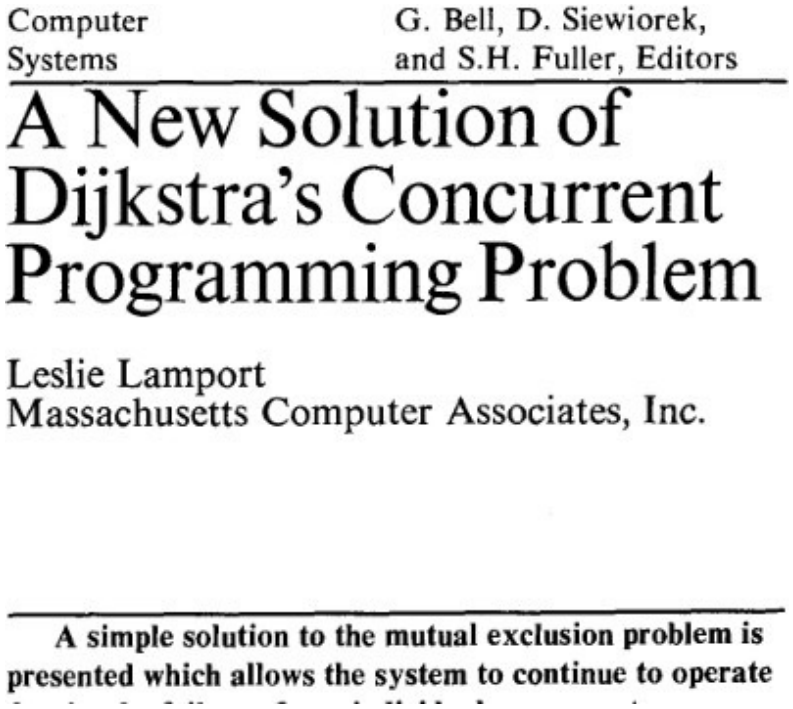
- "`if (no_note_A)`" happens before "`leave note_A`"



To formally prove things like this: look up "temporal logics of actions", e.g. TLA+

This generalizes to n threads…

- Leslie Lamport's "Bakery Algorithm" (1974)



Computer Systems — G. Bell, D. Siewiorek, and S.H. Fuller, Editors

A New Solution of Dijkstra's Concurrent Programming Problem

Leslie Lamport
Massachusetts Computer Associates, Inc.

A simple solution to the mutual exclusion problem is presented which allows the system to continue to operate

This generalizes to n threads…

- Solution #3 works, but it's really unsatisfactory
  - Really complex – even for this simple an example
    - Hard to convince yourself that this really works
  - A's code is different from B's – what if lots of threads?
    - Code would have to be slightly different for each thread
  - While A is waiting, it is consuming CPU time
    - This is called "busy-waiting"

Too much milk: solution #4?

- Recall our target lock interface:
  - `acquire(&milklock)` – wait until lock is free, then grab
  - `release(&milklock)` – Unlock, waking up anyone waiting
  - These must be atomic operations – if two threads are waiting for the lock and both see it's free, only one succeeds to grab the lock

- Then, our milk problem is easy:

```
acquire(&milk_lock);
if (no_milk)
  buy milk;
release(&milk_lock);
```

End of recall

How to implement locks?

- Prevents someone from doing something
- Lock before entering critical section and before accessing shared data
- Unlock when leaving, after accessing shared data

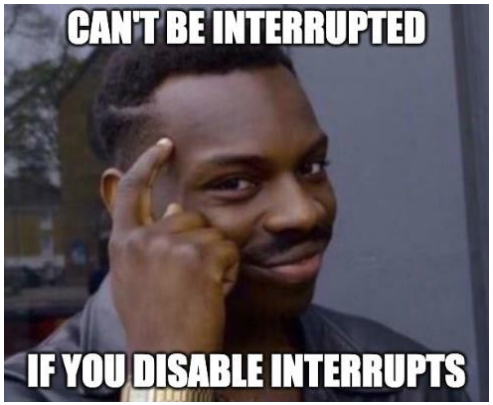


Hardware lock instruction?

- Is this a good idea?
- What about putting a task to sleep?
- What is the interface between the hardware and scheduler?
- Complexity?
  - Done in the Intel 432
  - Each feature makes HW more complex and slower

Lock implementation #1

How about disabling interrupts?



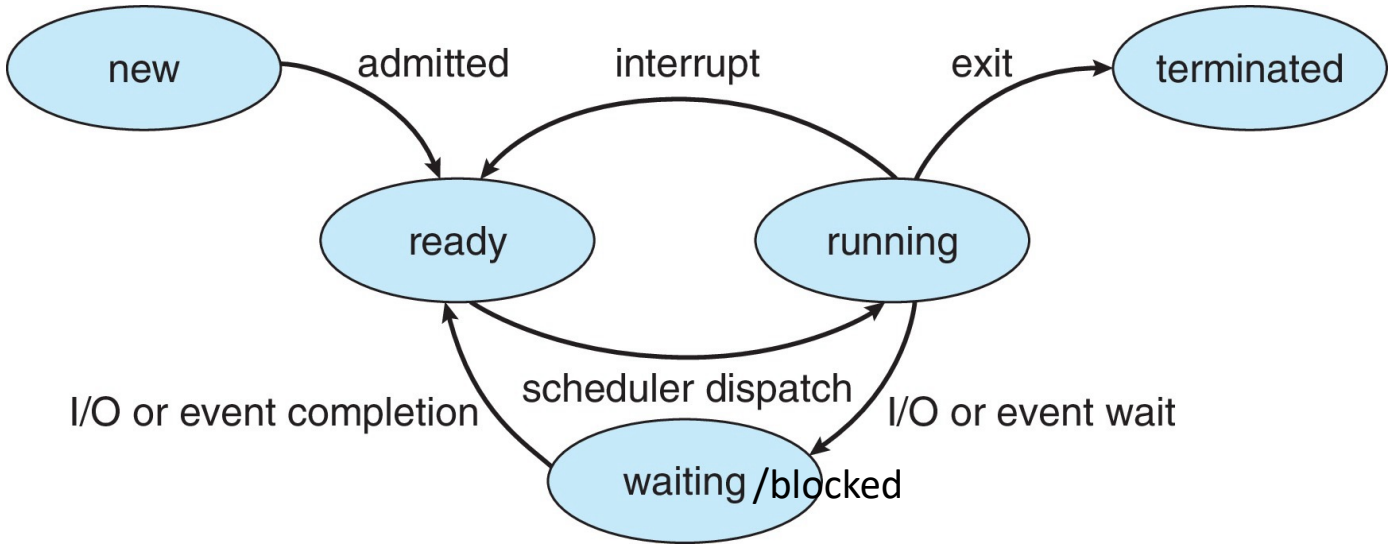
- Can we build multi-instruction atomic operations?

- Recall: dispatcher gets control in two ways.
  - Internal: thread does something to relinquish the CPU
  - External: interrupts cause dispatcher to take CPU

- On a uniprocessor, one can avoid context-switching by:
  - Avoiding internal events (although virtual memory is tricky)
  - Preventing external events by disabling interrupts

More details on thread state

- Thread state includes:
  - New: the thread is being created
  - Running: instructions are being executed
  - Waiting/Blocked: the thread is waiting for some event to occur
  - Ready: the thread is waiting to be assigned to a processor
  - Terminated: the tread has finished execution



How about disabling interrupts?

- Naïve implementation of locks:

```
LockAcquire { disable Ints; }

LockRelease { enable Ints; }
```

- Problems with this approach?

How about disabling interrupts?

- Consider the following:

```
LockAcquire();
while {;}
```

- Real-Time system—no guarantees on timing!
- Critical Sections might be arbitrarily long

- What happens with I/O or other important events?



Disabling interrupts – but smarter

- Key idea: maintain a lock variable and impose mutual exclusion only during operations on that variable



```
int value = FREE;

Acquire() {
  disable interrupts;
  if (value == BUSY) {
    put thread on wait queue;
    go to sleep();
    // Enable interrupts?
  } else {
    value = BUSY;
  }
  enable interrupts;
}
```

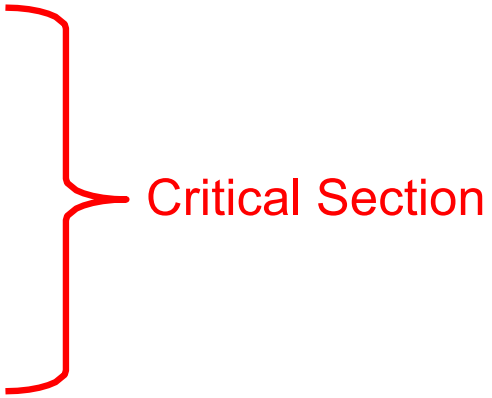
```
Release() {
  disable interrupts;
  if (anyone on wait queue) {
    take thread off wait queue;
    change thread to READY state;
  }
  value = FREE;
  enable interrupts;
}
```

- Note: `sleep()` is voluntarily giving up the CPU and entering the waiting/blocked state

How about disabling interrupts?

- Why do we need to disable interrupts at all?
  - Avoid interruption between checking and setting lock value
  - Otherwise two threads could think that they both have lock

```
Acquire() {
  disable interrupts;
  if (value == BUSY) {
    put thread on wait queue;
    go to sleep();
    // Enable interrupts?
  } else {
    value = BUSY;
  }
  enable interrupts;
}
```



- Note: unlike previous solution, the critical section (inside `Acquire()`) is very short

Interrupt re-enable in going to sleep

- What about re-enabling interrupts when going to sleep?

```
Acquire() {
  disable interrupts;
  if (value == BUSY) {
    put thread on wait queue;
    go to sleep();
  } else {
    value = BUSY;
  }
  enable interrupts;
}
```

Interrupt re-enable in going to sleep

- What about re-enabling interrupts when going to sleep?
  - Before putting thread on the wait queue?

Enable position

```
Acquire() {
  disable interrupts;
  if (value == BUSY) {
    put thread on wait queue;
    go to sleep();
  } else {
    value = BUSY;
  }
  enable interrupts;
}
```

- What if another thread releases lock immediately after enabling ints?
  - No thread on wait queue – no one will ever wake up this thread
  - Missed wakeup

Interrupt re-enable in going to sleep

- What about re-enabling interrupts when going to sleep?
  - After putting thread on the wait queue?

Enable position 

```
Acquire() {
  disable interrupts;
  if (value == BUSY) {
    put thread on wait queue;
    go to sleep();
  } else {
    value = BUSY;
  }
  enable interrupts;
}
```

- What if another thread releases lock immediately after enabling ints?
  - Thread is waken up before going to sleep – no other threads will wake it up
  - Missed wakeup

Interrupt re-enable in going to sleep

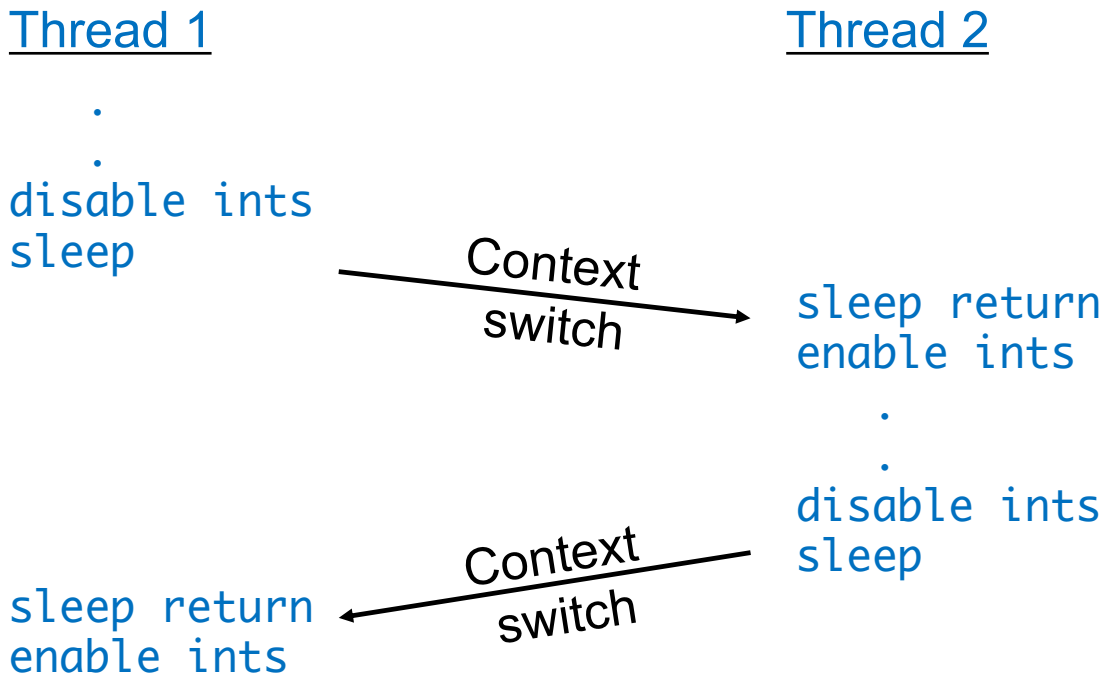
- What about re-enabling interrupts when going to sleep?
  - After sleep?

Enable position 

```
Acquire() {
  disable interrupts;
  if (value == BUSY) {
    put thread on wait queue;
    go to sleep();
  } else {
    value = BUSY;
  }
  enable interrupts;
}
```

How to re-enable interrupts after sleep()?

- In scheduler, since interrupts are disabled when you call sleep:
  - Responsibility of the next thread to re-enable interrupts
  - When the sleeping thread wakes up, returns to acquire and re-enables interrupts



What if there is no other thread?

- UNIX creates an “idle thread” at boot
- The idle thread never blocks
- So there is always another thread to help out

Lock implementation #2

Atomic read-modify-write instructions

- Problems with previous solution:
  - Can't give lock implementation to users
  - Doesn't work well on multiprocessor

- Alternative: atomic instruction sequences
  - These instructions read a value and write a new value atomically
  - Hardware is responsible for implementing this correctly
    - on both uniprocessors (not too hard)
    - and multiprocessors (requires help from cache coherence protocol)
  - Unlike disabling interrupts, this can be used on both uniprocessors and multiprocessors

Examples of read-modify-write

- ```
  test&set (&address) {
    result = M[address];
    M[address] = 1;
    return result;
  }
  ```

  ```
  /* most architectures */
  // return result from “address” and
  // set value at “address” to 1
  ```

- ```
  swap (&address, register) {
    temp = M[address];
    M[address] = register;
    register = temp;
  }
  ```

  ```
  /* x86 */
  // swap register’s value to
  // value at “address”
  ```

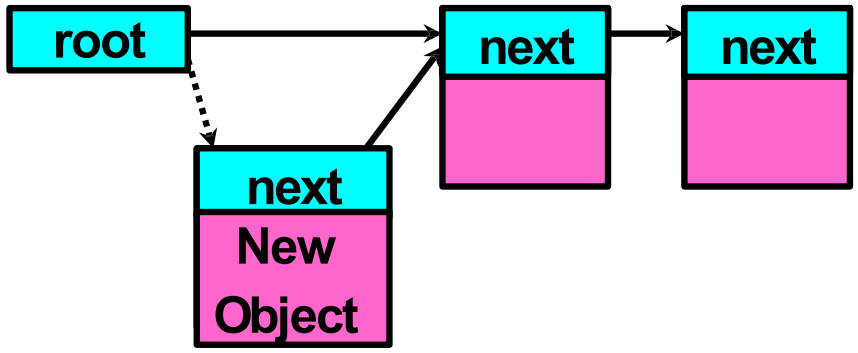
- ```
  compare&swap (&address, reg1, reg2) {
    if (reg1 == M[address]) {
      M[address] = reg2;
      return success;
    } else {
      return failure;
    }
  }
  ```

  ```
  /* x86 (returns old value), 68000 */
  // If memory still ==  reg1,
  // then put reg2 => memory

  // Otherwise do not change memory
  ```
  ```

Using compare&swap for queues

```
add_to_queue(&object) {
  do {                        // repeat until no conflict
    ld r1, M[root]            // get ptr to current head
    st r1, M[object]          // save link in new object
  } until (compare&swap(&root, r1, object));
}
```



Implementing locks with test&set

- Simple lock that doesn't require entry into the kernel

```
// Storage: just a memory location in userspace; 0 means "unlocked"
int mylock = 0;  // Interface: acquire(&mylock);
                 //            release(&mylock);

acquire(int *thelock) {
    while (test&set(thelock));    // Atomic operation!
}

release(int *thelock) {
    *thelock = 0;                 // Atomic operation!
}
```

Implementing locks with test&set

- Simple explanation:
  - If lock is free, test&set reads 0 and sets lock=1, so lock is now busy. It returns 0 so while exits.
  - If lock is busy, test&set reads 1 and sets lock=1 (no change). It returns 1, so while loop continues.
  - When we set thelock = 0 at release, someone else can get lock.

- Busy-waiting: thread consumes cycles while waiting
  - For multiprocessors: every test&set() is a write, which makes value ping-pong around in cache (using lots of network bandwidth)

Implementing locks with test&set

- Positives for this solution
  - Machine can receive interrupts
  - User code can use this lock
  - Works on a multiprocessor

- Negatives
  - This is very inefficient as thread will consume cycles waiting
  - Waiting thread may take cycles away from thread holding lock (no one wins!)
  - Homework/exam solutions should avoid busy-waiting!

Better locks using test&set

- Idea: only busy-wait to atomically check lock value



```
int guard = 0; // Global Variable!
int mylock = FREE; // Interface: acquire(&mylock);
                   //            release(&mylock);

acquire(int *thelock) {
  // Short busy-wait time
  while (test&set(guard));
  if (*thelock == BUSY) {
    put thread on wait queue;
    go to sleep() & guard = 0;
    // guard == 0 on wakeup!
  } else {
    *thelock = BUSY;
    guard = 0;
  }
}
```

```
release(int *thelock) {
  // Short busy-wait time
  while (test&set(guard));
  if anyone on wait queue {
    take thread off wait queue
    Place on ready queue;
  } else {
    *thelock = FREE;
  }
  guard = 0;
}
```

Linux futex: fast userspace mutex

```
#include <linux/futex.h>
#include <sys/time.h>

int futex(int *uaddr, int futex_op, int val,
          const struct timespec *timeout);
```

- `uaddr` points to a 32-bit value in user space
- `futex_op`
  - `FUTEX_WAIT` – if *`uaddr` == `val`, sleep till `FUTEX_WAKE`
    - ***Atomic*** check that condition still holds after we disable interrupts (in kernel!)
  - `FUTEX_WAKE` – wake up at most val waiting threads
  - `FUTEX_WAKE_OP`, `FUTEX_LOCK_PI`, `FUTEX_CMP_REQUEUE`: More interesting operations!
- `timeout`
  - ptr to a timespec structure that specifies a timeout for the op

Linux futex: fast userspace mutex

```
#include <linux/futex.h>
#include <sys/time.h>

int futex(int *uaddr, int futex_op, int val,
          const struct timespec *timeout);
```

- Interface to the kernel `sleep()` functionality!
  - Let thread put themselves to sleep – conditionally!

- `futex` is not exposed in libc; it is used within the implementation of pthreads
  - Can be used to implement locks, semaphores, monitors, etc…

Example: T&S and futex

```
int mylock = FREE; // Interface: acquire(&mylock);
                   //            release(&mylock);

acquire(int *thelock) {
  while (test&set(thelock)) {
    futex(thelock, FUTEX_WAIT, 1);
  }
}
```

```
release(int *thelock) {
  thelock = 0; // unlock
  futex(thelock, FUTEX_WAKE, 1);
}
```

- Sleep interface by using futex – no busy-waiting
- No kernel overhead to acquire lock
- Every unlock has to call kernel to potentially wake someone up – even if none

End of low-level lock implementation

Where are we going with synchronization?

Programs	Shared Programs	
Higher-level API	Locks Monitors	Semaphores Send/Receive
Hardware	Load/Store Test&Set	Disable Ints Compare&Swap

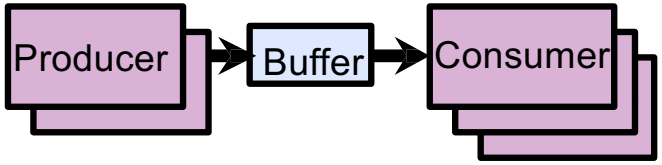
- We are going to implement various higher-level synchronization primitives using atomic operations
  - Everything is pretty painful if only atomic primitives are load and store
  - Need to provide primitives useful at user-level

Higher-level primitives than locks

- Goal of last couple of lectures:
  - What is right abstraction for synchronizing threads that share memory?
  - Want as high-level a primitive as possible

- Synchronization is a way of coordinating multiple concurrent activities that are using shared state
  - This lecture and the next present some ways of structuring sharing

Producer-consumer with a bounded buffer

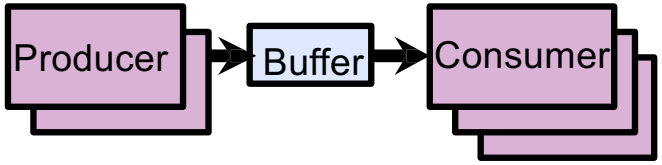


- Problem definition
  - Producer(s) put things into a shared buffer
  - Consumer(s) take them out
  - Need synchronization to coordinate producer/consumer

- Don't want producer and consumer to have to work in lockstep, so put a fixed-size buffer between them
  - Need to synchronize access to this buffer
  - Producer needs to wait if buffer is full
  - Consumer needs to wait if buffer is empty

Producer-consumer with a bounded buffer

- Example 1: GCCcompiler
  - `cpp | cc1 | cc2 | as | ld`

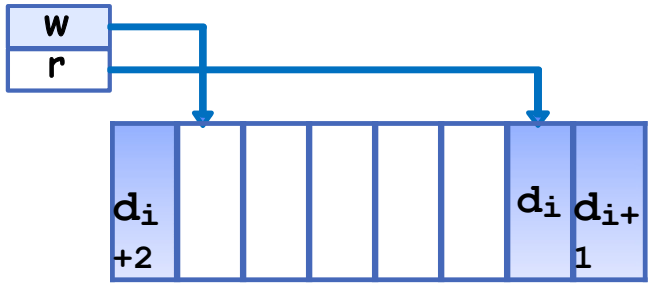


- Example 2: coke machine
  - Producer can put limited number of cokes in a machine
  - Consumer can't take cokes out if machine is empty

- Others: web servers, routers, …

Circular buffer data structure (sequential case)

```
typedef struct buf {
  int write_index;
  int read_index;
  <type> *entries[BUFSIZE];
} buf_t;
```



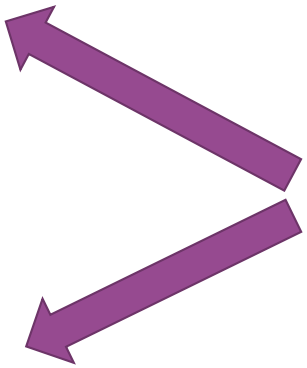
- Insert: write & bump write ptr (enqueue)
- Remove: read & bump read ptr (dequeue)
- How to tell if full (on insert) or empty (on remove)?
- And what do you do if it is?
- What needs to be atomic?

Circular buffer – first cut

```
Mutex buf_lock = <initially unlocked>

Producer(item) {
  acquire(&buf_lock);
  while (buffer full) {};  // Wait for a free slot
  enqueue(item);
  release(&buf_lock);
}

Consumer() {
  acquire(&buf_lock);
  while (buffer empty) {};
  item = dequeue();
  release(&buf_lock);
  return item;
}
```



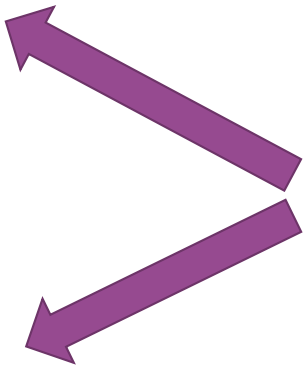
- Will we ever come out of the wait loop?

Circular buffer – second cut

```
Mutex buf_lock = <initially unlocked>

Producer(item) {
  acquire(&buf_lock);
  while (buffer full) { release(&buf_lock); acquire(&buf_lock); }
  enqueue(item);
  release(&buf_lock);
}

Consumer() {
  acquire(&buf_lock);
  while (buffer empty) { release(&buf_lock); acquire(&buf_lock); };
  item = dequeue();
  release(&buf_lock);
  return item;
}
```



- What happens when one is waiting for the other?
  - Multiple cores?
  - Single core?

Semaphores

- Semaphores are a type of generalized lock
- First defined by Dijkstra in late 60s
- Main synchronization primitive in original UNIX



Semaphores

- A Semaphore has a non-negative integer value and supports the following operations:
  - Set value when you initialize
  - Down() or P(): an atomic operation that waits for semaphore to become positive, then decrements it by 1
  - Up() or V(): an atomic operation that increments the semaphore by 1, waking up a waiting P, if any

Semaphores like integers except…

- Semaphores are like integers, except:
  - No negative values
  - Only operations allowed are P and V – can't read or write value, except initially
  - Operations must be atomic
    - Two P's together can't decrement value below zero
    - Thread going to sleep in P won't miss wakeup from V – even if both happen at the same time

Two uses of semaphores

- Mutual Exclusion (initial value = 1)
- Also called “Binary Semaphore” or “mutex”.
- Can be used for mutual exclusion, just like a lock:

```
semaP(&mysem);
// Critical section goes here
semaV(&mysem);
```

Two uses of semaphores

- Scheduling constraints (initial value = 0)
- Allow thread 1 to wait for a signal from thread 2
  - thread 2 schedules thread 1 when a given event occurs
- Suppose you had to implement ThreadJoin which must wait for thread to terminate

```
// Initial value of semaphore = 0
ThreadJoin {
  semaP(&mysem);
}
ThreadFinish {
  semaV(&mysem);
}
```

Bounded buffer: correctness constraints for solution

- Correctness constraints:
  - Consumer must wait for producer to fill buffers, if none full (scheduling constraint)
  - Producer must wait for consumer to empty buffers, if all full (scheduling constraint)
  - Only one thread can manipulate buffer queue at a time (mutual exclusion)

Bounded buffer: correctness constraints for solution

- General rule of thumb: use a separate semaphore for each constraint

  - Semaphore `full_buffers`; // consumer's constraint
  - Semaphore `empty_buffers`; // producer's constraint
  - Semaphore `mutex`; // mutual exclusion

Let's drink coke!




```
Semaphore full_slots = 0;          // Initially, no coke
Semaphore empty_slots = buf_size; // Initially, num empty slots
Semaphore mutex = 1;               // No one using machine

Producer(item) {
  semaP(&empty_slots);             // Wait until space




}
Consumer() {
  semaP(&full_slots);              // Check if there's a coke




}
```

Let's drink coke!




```
Semaphore full_slots = 0;          // Initially, no coke
Semaphore empty_slots = buf_size; // Initially, num empty slots
Semaphore mutex = 1;               // No one using machine

Producer(item) {
  semaP(&empty_slots);             // Wait until space

  enqueue(item);


}
Consumer() {
  semaP(&full_slots);              // Check if there's a coke

  item = dequeue();



}
```

Let's drink coke!




```
Semaphore full_slots = 0;          // Initially, no coke
Semaphore empty_slots = buf_size; // Initially, num empty slots
Semaphore mutex = 1;               // No one using machine

Producer(item) {
  semaP(&empty_slots);             // Wait until space

  enqueue(item);

  semaV(&full_slots);              // Tell consumers there is more coke
}
Consumer() {
  semaP(&full_slots);              // Check if there's a coke

  item = dequeue();

  semaV(&empty_slots);             // Tell producer need more
  return item
}
```

Let's drink coke!



```
Semaphore full_slots = 0;          // Initially, no coke
Semaphore empty_slots = buf_size; // Initially, num empty slots
Semaphore mutex = 1;               // No one using machine

Producer(item) {
  semaP(&empty_slots);             // Wait until space
  semaP(&mutex);                   // Wait until machine free
  enqueue(item);
  semaV(&mutex);
  semaV(&full_slots);              // Tell consumers there is more coke
}
Consumer() {
  semaP(&full_slots);              // Check if there's a coke
  semaP(&mutex);                   // Wait until machine free
  item = dequeue();
  semaV(&mutex);
  semaV(&empty_slots);             // Tell producer need more
  return item
}
```

Let's drink coke!



```
Semaphore full_slots = 0;          // Initially, no coke
Semaphore empty_slots = buf_size; // Initially, num empty slots
Semaphore mutex = 1;               // No one using machine

Producer(item) {
  semaP(&empty_slots);             // Wait until space
  semaP(&mutex);                   // Wait until machine free
  enqueue(item);
  semaV(&mutex);
  semaV(&full_slots);              // Tell consumers there is
}                                  // more coke
Consumer() {
  semaP(&full_slots);              // Check if there's a coke
  semaP(&mutex);                   // Wait until machine free
  item = dequeue();
  semaV(&mutex);
  semaV(&empty_slots);             // Tell producer need more
  return item
}
```

full_slots signal space

empty_slots signal space

Critical sections using mutex protect integrity of the queue

Discussion about solution

- Why asymmetry?
  - Producer does: **semaP(&empty_buffer), semaV(&full_buffer)**
  - Consumer does: **semaP(&full_buffer), semaV(&empty_buffer)**
  - Does order matter? What if we decrement mutex before full/empty_buffer?

Semaphores are good but…

- Semaphores are a huge step up; just think of trying to do the bounded buffer with only loads and stores or even with locks!

- Problem is that semaphores are dual purpose:
  - They are used for both mutex and scheduling constraints
  - Example: the fact that flipping of P's in bounded buffer gives deadlock is not immediately obvious. How do you prove correctness to someone?

Reading for next lecture

- Operating System Concepts:
  - Chapter 6.6-6.7 (6.4-6.6 if you want to review today's content)

Or

- Operating System: Three Easy Pieces:
  - Chapters 30, 31 and appendix "Monitors" (28-31 if you want to review today's content)

Thanks